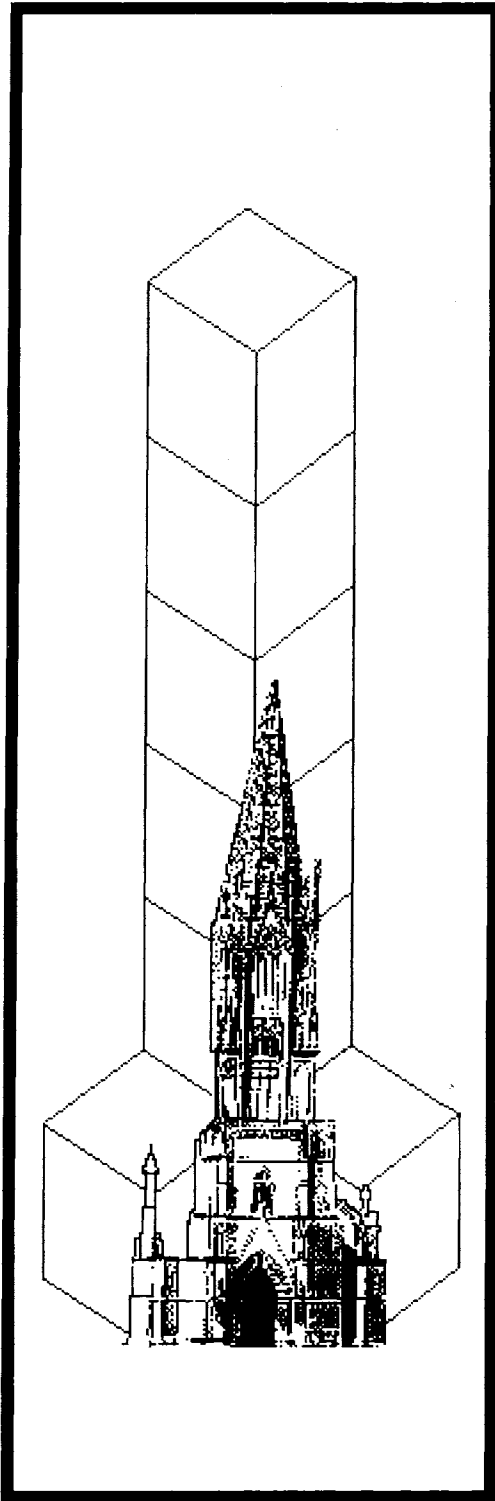




6

Protocol for All Objects

Testing the Functionality of an Object

Comparing Objects

Copying Objects

Accessing the Parts of an Object

Printing and Storing Objects

Error Handling

Object**Magnitude**

Character
Date
Time

Number

Float
Fraction
Integer
LargeNegativeInteger
LargePositiveInteger
SmallInteger

LookupKey

Association

Link**Process****Collection**

SequenceableCollection
LinkedList

Semaphore

ArrayedCollection
Array

Bitmap
DisplayBitmap

RunArray
String
Symbol

Text
ByteArray

Interval
OrderedCollection
SortedCollection

Bag
MappedCollection
Set

Dictionary
IdentityDictionary

Stream

PositionableStream
ReadStream
WriteStream
ReadWriteStream
ExternalStream
FileStream

Random

File
FileDirectory
FilePage

UndefinedObject

Boolean
False
True

ProcessorScheduler

Delay
SharedQueue

Behavior

ClassDescription
Class
MetaClass

Point
Rectangle
BitBit

CharacterScanner

Pen**DisplayObject**

DisplayMedium
Form
Cursor
DisplayScreen
InfiniteForm
OpaqueForm
Path

Arc
Circle
Curve
Line
LinearFit
Spline

Everything in the system is an object. The protocol common to all objects in the system is provided in the description of class `Object`. This means that any and every object created in the system can respond to the messages defined by class `Object`. These are typically messages that support reasonable default behavior in order to provide a starting place from which to develop new kinds of objects, either by adding new messages or by modifying the response to existing messages. Examples to consider when examining `Object`'s protocol are numeric objects such as 3 or 16.23, collections such as 'this is a string' or #(this is an array), nil or true, and class-describing objects such as `Collection` or `SmallInteger` or, indeed, `Object` itself.

The specification of protocol for class `Object` given in this chapter is incomplete. We have omitted messages pertaining to message handling, special dependency relationships, and system primitives. These are presented in Chapter 14.

Testing the
Functionality
of an Object

Every object is an instance of a class. An object's functionality is determined by its class. This functionality is tested in two ways: explicit naming of a class to determine whether it is the class or the superclass of the object, and naming of a message selector to determine whether the object can respond to it. These reflect two ways of thinking about the relationship among instances of different classes: in terms of the class/subclass hierarchy, or in terms of shared message protocols.

Object instance protocol

testing functionality

class	Answer the object which is the receiver's class.
isKindOf: aClass	Answer whether the argument, aClass, is a superclass or class of the receiver.
isMemberOf: aClass	Answer whether the receiver is a direct instance of the argument, aClass. This is the same as testing whether the response to sending the receiver the message class is the same as (==) aClass.
respondsTo: aSymbol	Answer whether the method dictionary of the receiver's class or one of its superclasses contains the argument, aSymbol, as a message selector.

Example messages and their corresponding results are

<i>expression</i>	<i>result</i>
3 class	SmallInteger
#(this is an array) isKindOf: Collection	true

#(this is an array) isMemberOf: Collection	false
#(this is an array) class	Array
3 respondsTo: #isKindOf:	true
#(1 2 3) isMemberOf: Array	true
Object class	Object class

Comparing Objects

Since all information in the system is represented as objects, there is a basic protocol provided for testing the identity of an object and for copying objects. The important comparisons specified in class `Object` are equivalence and equality testing. Equivalence (`==`) is the test of whether two objects are the same object. Equality (`=`) is the test of whether two objects represent the same component. The decision as to what it means to be "represent the same component" is made by the receiver of the message; each new kind of object that adds new instance variables typically must reimplement the `=` message in order to specify which of its instance variables should enter into the test of equality. For example, equality of two arrays is determined by checking the size of the arrays and then the equality of each of the elements of the arrays; equality of two numbers is determined by testing whether the two numbers represent the same value; and equality of two bank accounts might rest solely on the equality of each account identification number.

The message `hash` is a special part of the comparing protocol. The response to `hash` is an integer. Any two objects that are equal must return the same value for `hash`. Unequal objects may or may not return equal values for `hash`. Typically, this integer is used as an index to locate the object in an indexed collection (as illustrated in Chapter 3). Any time `=` is redefined, `hash` may also have to be redefined in order to preserve the property that any two objects that are equal return equal values for `hash`.

Object instance protocol ---

comparing

`== anObject`

Answer whether the receiver and the argument are the same object.

`= anObject`

Answer whether the receiver and the argument represent the same component.

`~= anObject`

Answer whether the receiver and the argument do not represent the same component.

`~~ anObject`

Answer whether the receiver and the argument are not the same object.

`hash`

Answer an Integer computed with respect to the representation of the receiver.

The default implementation of `=` is the same as that of `==`.

Some specialized comparison protocol provides a concise way to test for identity with the object `nil`.

Object instance protocol

testing

<code>isNil</code>	Answer whether the receiver is <code>nil</code> .
<code>notNil</code>	Answer whether the receiver is not <code>nil</code> .

These messages are identical to `== nil` and `~~ nil`, respectively. Choice of which to use is a matter of personal style.

Some obvious examples are

<i>expression</i>	<i>result</i>
<code>nil isNil</code>	<code>true</code>
<code>true notNil</code>	<code>true</code>
<code>3 isNil</code>	<code>false</code>
<code>#(a b c) = #(a b c)</code>	<code>true</code>
<code>3 = (6/2)</code>	<code>true</code>
<code>#(1 2 3) class == Array</code>	<code>true</code>

Copying Objects

There are two ways to make copies of an object. The distinction is whether or not the values of the object's variables are copied. If the values are not copied, then they are shared (`shallowCopy`); if the values are copied, then they are not shared (`deepCopy`).

Object instance protocol

copying

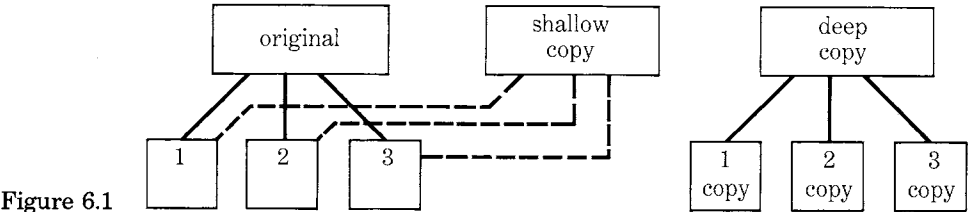
<code>copy</code>	Answer another instance just like the receiver.
<code>shallowCopy</code>	Answer a copy of the receiver which shares the receiver's instance variables.
<code>deepCopy</code>	Answer a copy of the receiver with its own copy of each instance variable.

The default implementation of `copy` is `shallowCopy`. In subclasses in which copying must result in a special combination of shared and unshared variables, the method associated with `copy` is usually re-implemented, rather than the method associated with `shallowCopy` or `deepCopy`.

As an example, a copy (a shallow copy) of an Array refers to the same elements as in the original Array, but the copy is a different object. Replacing an element in the copy does not change the original. Thus

<i>expression</i>	<i>result</i>
<code>a ← #('first' 'second' 'third')</code>	<code>('first' 'second' 'third')</code>
<code>b ← a copy</code>	<code>('first' 'second' 'third')</code>
<code>a = b</code>	<code>true</code>
<code>a == b</code>	<code>false</code>
<code>(a at: 1) == (b at: 1)</code>	<code>true</code>
<code>b at: 1 put: 'newFirst'</code>	<code>'newFirst'</code>
<code>a = b</code>	<code>false</code>
<code>a ← 'hello'</code>	<code>'hello'</code>
<code>b ← a copy</code>	<code>'hello'</code>
<code>a = b</code>	<code>true</code>
<code>a == b</code>	<code>false</code>

Figure 6.1 shows the relationship between shallow and deep copying. To further illustrate the distinction between shallowCopy and deepCopy, take as an example a PersonnelRecord. Suppose it is defined to include the variable insurancePlan, an instance of class Insurance. Suppose further that each instance of Insurance has a value associated with it representing the limit on medical coverage. Now suppose we have created employeeRecord as a prototypical instance of a PersonnelRecord. By “prototypical” we mean that the object has all of the initial attributes of any new instance of its class, so that instances can be created by simply copying it rather than sending a sequence of initialization messages. Suppose further that this prototypical instance is a class variable of PersonnelRecord and that the response to creating a new PersonnelRecord is to make a shallow copy of it; that is, the method associated with the message new is ↑employeeRecord copy.



As a result of evaluating the expression

`joeSmithRecord ← PersonnelRecord new`

joeSmithRecord refers to a copy (in particular, a shallow copy) of employeeRecord.

The prototype employeeRecord and the actual record joeSmithRecord share a reference to the same insurance plan. Company policy may change. Suppose PersonnelRecord understands the message changeInsuranceLimit: aNumber, which is implemented by having the prototypical instance of PersonnelRecord, employeeRecord, reset its insurance plan limit on medical coverage. Since this insurance plan is shared, the result of evaluating the expression

PersonnelRecord changeInsuranceLimit: 4000

is to change the medical coverage of all employees. In the example, both the medical coverage referenced by employeeRecord and that referenced by its copy, joeSmithRecord, is changed. The message changeInsuranceLimit: is sent to the class PersonnelRecord because it is the appropriate object to broadcast a change to all of its instances.

Accessing the Parts of an Object

There are two kinds of objects in the Smalltalk-80 system, objects with named variables and objects with indexed variables. Objects with indexed variables may also have named instance variables. This distinction is explained in Chapter 3. Class Object supports six messages intended to access the indexed variables of an object. These are

Object instance protocol

accessing

at: index

Answer the value of the indexed instance variable of the receiver whose index is the argument, index. If the receiver does not have indexed variables, or if the argument is greater than the number of indexed variables, then report an error.

at: index put: anObject

Store the argument, anObject, as the value of the indexed instance variable of the receiver whose index is the argument, index. If the receiver does not have indexed variables, or if the argument is greater than the number of indexed variables, then report an error. Answer anObject.

basicAt: index

Same as at: index. The method associated with this message, however, cannot be modified in any subclass.

<code>basicAt: index put: anObject</code>	Same as <code>at: index put: anObject</code> . The method associated with this message, however, cannot be modified in any subclass.
<code>size</code>	Answer the receiver's number of indexed variables. This value is the same as the largest legal index.
<code>basicSize</code>	Same as <code>size</code> . The method associated with this message, however, cannot be modified in any subclass.

Notice that the accessing messages come in pairs; one message in each pair is prefixed by the word `basic` meaning that it is a fundamental system message whose implementation should not be modified in any subclass. The purpose of providing pairs is so that the external protocol, `at:`, `at:put:`, and `size`, can be overridden to handle special cases, while still maintaining a way to get at the primitive methods. (Chapter 4 includes an explanation of "primitive" methods, which are methods implemented in the virtual machine for the system.) Thus in any method in a hierarchy of class descriptions, the messages, `basicAt:`, `basicAt:put:`, and `basicSize`, can always be used to obtain the primitive implementations. The message `basicSize` can be sent to any object; if the object is not variable length, then the response is 0.

Instances of class `Array` are variable-length objects. Suppose `letters` is the `Array` `#(a b d f j m p s)`. Then

<i>expression</i>	<i>result</i>
<code>letters size</code>	8
<code>letters at: 3</code>	d
<code>letters at: 3 put: #c</code>	c
<code>letters</code>	(a b c f j m p s)

Printing and Storing Objects

There are various ways to create a sequence of characters that provides a description of an object. The description might give only a clue as to the identity of an object. Or the description might provide enough information so that a similar object can be constructed. In the first case (printing), the description may or may not be in a well-formatted, visually pleasing style, such as that provided by a Lisp pretty-printing routine. In the second case (storing), the description might preserve information shared with other objects.

The message protocol of the classes in the Smalltalk-80 system support printing and storing. The implementation of these messages in class `Object` provides minimal capability; most subclasses override the

messages in order to enhance the descriptions created. The arguments to two of the messages are instances of a kind of Stream; Streams are presented in Chapter 12.

Object instance protocol

printing

printString

Answer a String whose characters are a description of the receiver.

printOn: aStream

Append to the argument, aStream, a String whose characters are a description of the receiver.

storing

storeString

Answer a String representation of the receiver from which the receiver can be reconstructed.

storeOn: aStream

Append to the argument, aStream, a String representation of the receiver from which the receiver can be reconstructed.

Each of the two kinds of printing is based on producing a sequence of characters that may be shown on a display screen, written on a file, or transferred over a network. The sequence created by storeString or storeOn: should be interpretable as one or more expressions that can be evaluated in order to reconstruct the object. Thus, for example, a Set of three elements, \$a, \$b, and \$c, might print as

Set (\$a \$b \$c)

while it might store as

(Set new add: \$a; add: \$b; add: \$c)

Literals can use the same representation for printing and storing. Thus the String 'hello' would print and store as 'hello'. The Symbol #name prints as name, but stores as #name.

For lack of more information, the default implementation of printString is the object's class name; the default implementation of storeString is the class name followed by the instance creation message basicNew, followed by a sequence of messages to store each instance variable. For example, if a subclass of Object, say class Example, demonstrated the default behavior, then, for eg, an instance of Example with no instance variables, we would have

<i>expression</i>	<i>result</i>
eg printString	'an Example'
eg storeString	'(Example basicNew)'

Error Handling

The fact that all processing is carried out by sending messages to objects means that there is one basic error condition that must be handled by the system: a message is sent to an object, but the message is not specified in any class in the object's superclass chain. This error is determined by the interpreter whose reaction is to send the original object the message `doesNotUnderstand: aMessage`. The argument, `aMessage`, represents the failed message selector and its associated arguments, if any. The method associated with `doesNotUnderstand:` gives the user a report that the error occurred. How the report is presented to the user is a function of the (graphical) interface supported by the system and is not specified here; a minimum requirement of an interactive system is that the error message be printed on the user's output device and then the user be given the opportunity to correct the erroneous situation. Chapter 17 illustrates the Smalltalk-80 system error notification and debugging mechanisms.

In addition to the basic error condition, methods might explicitly want to use the system error handling mechanism for cases in which a test determines that the user program is about to do something unacceptable. In such cases, the method might want to specify an error comment that should be presented to the user. A typical thing to do is to send the active instance the message `error: aString`, where the argument represents the desired comment. The default implementation is to invoke the system notification mechanism. The programmer can provide an alternative implementation for `error:` that uses application-dependent error reporting.

Common error messages are supported in the protocol of class `Object`. An error message might report that a system primitive failed, or that a subclass is overriding an inherited message which it can not support and therefore the user should not call upon it, or that a superclass specifies a message that must be implemented in a subclass.

Object instance protocol

error handling

<code>doesNotUnderstand: aMessage</code>	Report to the user that the receiver does not understand the argument, <code>aMessage</code> , as a message.
<code>error: aString</code>	Report to the user that an error occurred in the context of responding to a message to the receiver. The report uses the argument, <code>aString</code> , as part of the error notification comment.
<code>primitiveFailed</code>	Report to the user that a method implemented as a system primitive has failed.

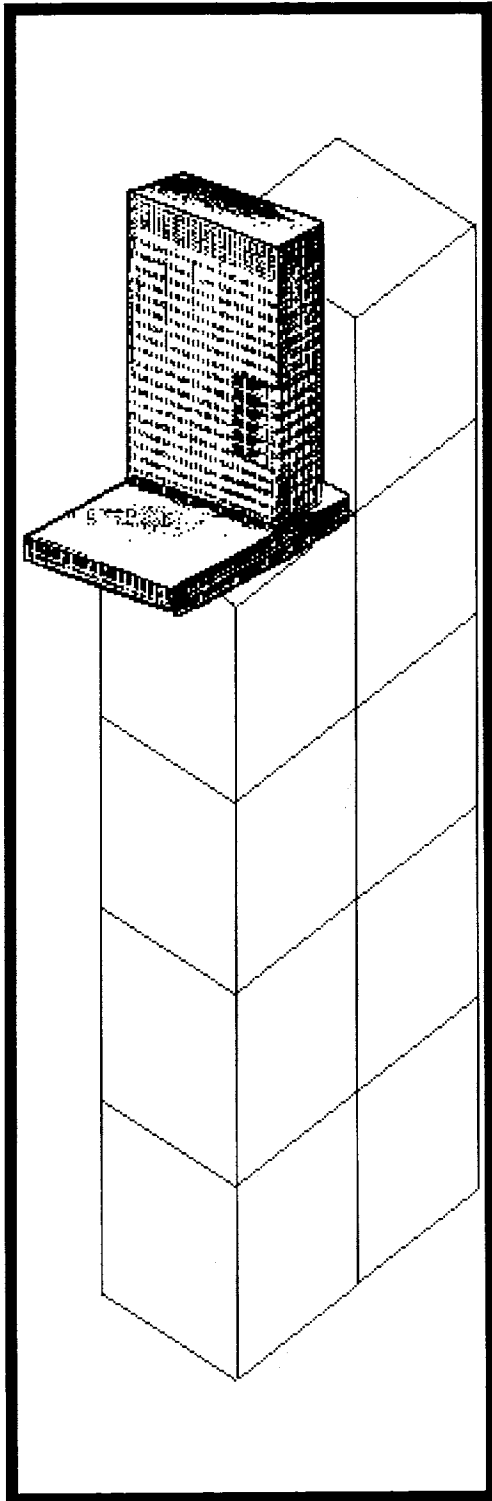
`shouldNotImplement`

Report to the user that, although the superclass of the receiver specifies that a message should be implemented by subclasses, the class of the receiver cannot provide an appropriate implementation.

`subclassResponsibility`

Report to the user that a method specified in the superclass of the receiver should have been implemented in the receiver's class.

A subclass can choose to override the error-handling messages in order to provide special support for correcting the erroneous situation. Chapter 13, which is about the implementation of the collection classes, provides examples of the use of the last two messages.



7

Linear Measures

Class Magnitude

Class Date

Class Time

Class Character

Object

Magnitude
Character
Date
Time

Number
Float
Fraction
Integer
LargeNegativeInteger
LargePositiveInteger
SmallInteger

LookupKey
Association

Link

Process

Collection

SequenceableCollection
LinkedList

Semaphore

ArrayedCollection
Array

Bitmap
DisplayBitmap

RunArray
String
Symbol

Text
ByteArray

Interval
OrderedCollection
SortedCollection

Bag
MappedCollection
Set

Dictionary
IdentityDictionary

Stream

PositionableStream
ReadStream
WriteStream
ReadWriteStream
ExternalStream
FileStream

Random

File
FileDirectory
FilePage

UndefinedObject

Boolean
False
True

ProcessorScheduler

Delay
SharedQueue

Behavior

ClassDescription
Class
MetaClass

Point

Rectangle

BitBit

CharacterScanner

Pen

DisplayObject

DisplayMedium
Form
Cursor
DisplayScreen

InfiniteForm
OpaqueForm

Path

Arc
Circle
Curve
Line
LinearFit
Spline

The Smalltalk-80 system provides several classes representing objects that measure something with linear ordering. Real world examples of such measurable quantities are (1) temporal quantities such as dates and time, (2) spatial quantities such as distance, and (3) numerical quantities such as reals and rationals.

Class Magnitude

Is one number less than another number? Does one date come after another date? Does one time precede another time? Does a character come after another one in the alphabet? Is one distance the same or less than another distance?

The common protocol for answering these queries is provided in the class `Magnitude`. `Magnitude` provides the protocol for objects that have the ability to be compared along a linear dimension. Subclasses of class `Magnitude` include `Date`, `Time`, and `Number`. Classes `Character` (an element of a string) and `LookupKey` (a key in a dictionary association) are also implemented as subclasses of class `Magnitude`. `Character` is interesting as an example of immutable objects in the system and so is introduced in this chapter; `LookupKey` is less interesting and is deferred until needed in the chapter on collections. A class `Distance` is not provided in the actual Smalltalk-80 system.

Magnitude instance protocol

comparing

<code>< aMagnitude</code>	Answer whether the receiver is less than the argument.
<code><= aMagnitude</code>	Answer whether the receiver is less than or equal to the argument.
<code>> aMagnitude</code>	Answer whether the receiver is greater than the argument.
<code>>= aMagnitude</code>	Answer whether the receiver is greater than or equal to the argument.
<code>between: min and: max</code>	Answer whether the receiver is greater than or equal to the argument, min, and less than or equal to the argument, max.

Although `Magnitude` inherits from its superclass, `Object`, the message `=` for comparing the equality of two quantifiable objects, every kind of `Magnitude` must redefine this message. The method associated with `=` in class `Magnitude` is

self subclassResponsibility

If a subclass of `Magnitude` does not implement `=`, then an attempt to send the message to an instance of the subclass results in the special error message that a subclass should have implemented the message, as specified in its superclass.

An instance of a kind of Magnitude can also respond to messages that determine which of two objects that can be linearly measured is the larger or the smaller.

Magnitude instance protocol

testing

min: aMagnitude	Answer the receiver or the argument, whichever has the lesser magnitude.
max: aMagnitude	Answer the receiver or the argument, whichever has the greater magnitude.

Note that protocol for the equality comparisons `=`, `~`, and `~~` is inherited from class `Object`. Using `Integers` as the example kinds of `Magnitudes`, we have

<i>expression</i>	<i>result</i>
<code>3 <= 4</code>	true
<code>3 > 4</code>	false
<code>5 between: 2 and: 6</code>	true
<code>5 between: 2 and: 4</code>	false
<code>34 min: 45</code>	34
<code>34 max: 45</code>	45

The programmer does not create instances of `Magnitude`, but only of its subclasses. This is due to the fact that `Magnitude` is not able to implement all of the messages it specifies, indeed, that it implements one or more of these messages by the expression `self subclassResponsibility`.

Class Date

Now that we have defined the general protocol of `Magnitudes`, it is possible to add additional protocol that supports arithmetic and inquiries about specific linear measurements. The first refinement we will examine is the subclass `Date`.

An instance of `Date` represents a specific day since the start of the Julian calendar. A day exists in a particular month and year. Class `Date` knows about some obvious information: (1) there are seven days in a week, each day having a symbolic name and an index 1, 2, ..., or 7; (2) there are 12 months in a year, each having a symbolic name and an index, 1, 2, ..., or 12; (3) months have 28, 29, 30, or 31 days; and (4) a particular year might be a leap year.

Protocol provided for the object, `Date`, supports inquiries about `Dates` in general as well as about a specific `Date`. Both `Date` and `Time` provide interesting examples of classes in the system for which special knowledge is attributed to and accessible from the class itself, rather than

from its instances. This “class protocol” is specified in the metaclass of the class. Let’s first look at the class protocol of Date supporting general inquiries.

Date class protocol

general inquiries

dayOfWeek: dayName	Answer the index in a week, 1, 2, ..., or 7, of the day named as the argument, dayName.
nameOfDay: dayIndex	Answer a Symbol that represents the name of the day whose index is the argument, dayIndex, where 1 is Monday, 2, is Tuesday, and so on.
indexOfMonth: monthName	Answer the index in a year, 1, 2, ..., or 12, of the month named as the argument, monthName.
nameOfMonth: monthIndex	Answer a Symbol that represents the name of the month whose index is the argument, monthIndex, where 1 is January, 2, is February, and so on.
daysInMonth: monthName forYear: yearInteger	Answer the number of days in the month whose name is monthName in the year yearInteger (the year must be known in order to account for a leap year).
daysInYear: yearInteger	Answer the number of days in the year, yearInteger.
leapYear: yearInteger	Answer 1 if the year yearInteger is a leap year; answer 0 otherwise.
dateAndTimeNow	Answer an Array whose first element is the current date (an instance of class Date representing today’s date) and whose second element is the current time (an instance of class Time representing the time right now).

Thus we can send the following messages.

<i>expression</i>	<i>result</i>
Date daysInYear: 1982	365
Date dayOfWeek: #Wednesday	3
Date nameOfMonth: 10	October
Date leapYear: 1972	1 (meaning it is a leap year)
Date daysInMonth: #February forYear: 1972	29
Date daysInMonth: #Feb forYear: 1971	28

Date is familiar with the common abbreviations for names of months.

There are four messages that can be used to create an instance of class `Date`. The one commonly used in the Smalltalk-80 system, notably for marking the creation date of a file, is `Date today`.

Date class protocol

instance creation

<code>today</code>	Answer an instance of <code>Date</code> representing the day the message is sent.
<code>fromDays: dayCount</code>	Answer an instance of <code>Date</code> that is <code>dayCount</code> number of days before or after January 1, 1901 (depending on the sign of the argument).
<code>newDay: day month: monthName year: yearInteger</code>	Answer an instance of <code>Date</code> that is <code>day</code> number of days into the month named <code>monthName</code> in the year <code>yearInteger</code> .
<code>newDay: dayCount year: yearInteger</code>	Answer an instance of <code>Date</code> that is <code>dayCount</code> number of days after the beginning of the year <code>yearInteger</code> .

Four examples of instance creation messages are

<i>expression</i>	<i>result</i>
<code>Date today</code>	3 February 1982
<code>Date fromDays: 200</code>	20 July 1901
<code>Date newDay: 6 month: #Feb year: 82</code>	6 February 1982
<code>Date newDay: 3 year: 82</code>	3 January 1982

Messages that can be sent to an instance of `Date` are categorized as accessing, inquiries, arithmetic, and printing messages. Accessing and inquiries about a particular day consist of

- the day index, month index, or year
- the number of seconds, days, or months since some other date
- the total days in the date's month or year
- the days left in the date's month or year
- the first day of the date's month
- the name of the date's weekday or month
- the date of a particular weekday previous to the instance

Simple arithmetic is supported in the protocol of class `Date`.

Date instance protocol

arithmetic

addDays: dayCount	Answer a Date that is dayCount number of days after the receiver.
subtractDays: dayCount	Answer a Date that is dayCount number of days before the receiver.
subtractDate: aDate	Answer an Integer that represents the number of days between the receiver and the argument, aDate.

Such arithmetic is useful, for example, in order to compute due dates for books in a library or fines for late books. Suppose dueDate is an instance of Date denoting the day a book was supposed to be returned to the library. Then

Date today subtractDate: dueDate

computes the number of days for which the borrower should be fined. If a book is being borrowed today and it can be kept out for two weeks, then

Date today addDays: 14

is the due date for the book. If the librarian wants to quit work 16 days before Christmas day, then the date of the last day at work is

(Date newDay: 25 month: #December year: 1982) subtractDays: 16

An algorithm to determine the fine a borrower must pay might first compare today's date with the due date and then, if the due date has past, determine the fine as a 10-cent multiple of the number of days overdue.

```
Date today < dueDate
  ifTrue: [fine ← 0]
  ifFalse: [fine ← 0.10 * (Date today subtractDate: dueDate)]
```

Class Time

An instance of class Time represents a particular second in a day. Days start at midnight. Time is a subclass of Magnitude. Like class Date, Time can respond to general inquiry messages that are specified in the class protocol.

Time class protocol

general inquiries

millisecondClockValue	Answer the number of milliseconds since the millisecond clock was last reset or rolled over to 0.
millisecondsToRun: timedBlock	Answer the number of milliseconds timedBlock takes to return its value.
timeWords	Answer the seconds (in Greenwich Mean Time) since Jan. 1, 1901. The answer is a four-element ByteArray (ByteArray is described in Chapter 10).
totalSeconds	Answer the total seconds from Jan. 1, 1901, corrected for time zone and daylight savings time.
dateAndTimeNow	Answer an Array whose first element is the current date (an instance of class Date that represents today's date) and whose second element is the current time (an instance of class Time that represents the time right now). The result of sending this message to Time is identical to the result of sending it to Date.

The only non-obvious inquiry is `millisecondsToRun: timedBlock`. An example is

Time millisecondsToRun: [Date today]

where the result is the number of milliseconds it took the system to compute today's date. Because there is some overhead in responding to this message, and because the resolution of the clock affects the result, the careful programmer should determine the machine-dependent uncertainties associated with selecting reasonable arguments to this message.

A new instance of Time can be created by sending Time the message `now`; the corresponding method reads the current time from a system clock. Alternatively, an instance of Time can be created by sending the message `fromSeconds: secondCount`, where `secondCount` is the number of seconds since midnight.

Time class protocol

instance creation

now	Answer an instance of Time representing the second the message is sent.
fromSeconds: secondCount	Answer an instance of Time that is secondCount number of seconds since midnight.

Accessing protocol for instances of class Time provide information as to the number of hours (hours), minutes (minutes) and seconds (seconds)

that the instance represents.
Arithmetic is also supported.

Time instance protocol

arithmetic

addTime: timeAmount	Answer an instance of Time that is the argument, timeAmount, after the receiver.
subtractTime: timeAmount	Answer an instance of Time that is the argument, timeAmount, before the receiver.

In the messages given above, the arguments (timeAmount) may be either Dates or Times. For this to be possible, the system must be able to convert a Date and a Time to a common unit of measurement; it converts them to seconds. In the case of Time, the conversion is to the number of seconds since midnight; in the case of Date, the conversion is to the number of seconds between a time on January 1, 1901, and the same time in the receiver's day. To support these methods, instances of each class respond to the conversion message asSeconds.

Time instance protocol

converting

asSeconds	Answer the number of seconds since midnight that the receiver represents.
-----------	---

Date instance protocol

converting

asSeconds	Answer the number of seconds between a time on January 1, 1901, and the same time in the receiver's day.
-----------	--

Arithmetic for Time can be used in ways analogous to that for Date. Suppose the amount of time a person spends working on a particular project is to be logged so that a customer can be charged an hourly fee. Suppose the person started work at startTime and worked continuously during the day until right now; the phone rings and the customer wants to know today's charges. At that moment, the bill at \$5.00 an hour is

$(\text{Time now subtractTime: startTime}) \text{ hours} * 5$

ignoring any additional minutes or seconds. If a charge for any fraction of an hour over 30 minutes is to be charged as a full hour then an additional \$5.00 is added if

$(\text{Time now subtractTime: startTime}) \text{ minutes} > 30$

Suppose times are computed across days, for example, in computing the time of a car in a four-day rally. If the first day of the rally started at `startTime` on day `startDate`, then the time for a car arriving at the finish line right now is computed as follows.

```
startTime ← Time fromSeconds: (60*60*6).
```

```
startDate ← Date newDay: 2 month: #Feb year: 82.
```

```
todayStart ← (((Time fromSeconds: 0) addTime: Date today)
              subtractTime: startDate)
              subtractTime: startTime
```

```
(Date today subtractDate: startDate) * 24*60*60)
```

todayStart addTime: Time now

Class **Character** is the third subclass of class **Magnitude** we shall examine. It is a kind of **Magnitude** because instances of class **Character** form an ordered sequence about which we can make inquiries such as whether one character precedes (<) or succeeds (>) another character alphabetically. There are 256 instances of class **Character** in the system. Each one is associated with a code in an extended ASCII character set.

Characters can be expressed literally by preceding the alphabetic character by a dollar sign (\$); thus, \$A is the Character representing the capital letter "A". Protocol for creating instances of class Character consists of

Character class protocol

instance creation

value: anInteger

Answer an instance of Character whose value is the argument, anInteger. The value is associated with an element of the ASCII character set. For example, Character value: 65 is a capital "A".

digitValue: anInteger

Answer an instance of Character whose digit value is the argument, anInteger. For example, answer \$9 if the argument is 9; answer \$0 for 0; answer \$A for 10, and \$Z for 35. This method is useful in parsing numbers into strings. Typically, only Characters up to \$F are useful (for base 16 numbers).

Class protocol, that is, the set of messages to the object Character, provides a vocabulary for accessing characters that are not easy to distinguish when printed: backspace, cr, esc, newPage (that is, form feed), space, and tab.

Messages to instances of Character support accessing the ASCII value and the digit value of the instance and testing the type of character. The only state of a Character is its value which can never change. Objects that can not change their internal state are called *immutable objects*. This means that, once created, they are not destroyed and then recreated when they are needed again. Rather, the 256 instances of Character are created at the time the system is initialized and remain in the system. Whenever a new Character whose code is between 0 and 255 is requested, a reference is provided to an already existing Character. In this way the 256 Characters are unique. Besides Characters, the Smalltalk-80 system includes SmallIntegers and Symbols as immutable objects.

Character instance protocol

accessing

asciiValue

Answer the number corresponding to the ASCII encoding for the receiver.

digitValue

Answer the number corresponding to the numerical radix represented by the receiver (see the instance creation message digitValue: for the correspondences).

testing

isAlphaNumeric

Answer true if the receiver is a letter or a digit.

isDigit

Answer whether the receiver is a digit.

isLetter

Answer whether the receiver is a letter.

isLowercase

Answer whether the receiver is a lowercase letter.

isUppercase

Answer whether the receiver is an uppercase letter.

isSeparator

Answer whether the receiver is one of the separator characters in the expression syntax: space, cr, tab, line feed, or form feed.

isVowel

Answer whether the receiver is one of the vowels: a, e, i, o, or u, in upper or lowercase.

Instance protocol also provides conversion of a character into upper- or lowercase (**asLowercase** and **asUppercase**) and into a symbol (**asSymbol**).

A simple alphabetic comparison demonstrates the use of comparing protocol for instances of **Character**. Suppose we wish to know if one string of characters precedes another string in the telephone book. **Strings** respond to the message **at:** to retrieve the element whose index is the argument; elements of **Strings** are **Characters**. Thus 'abc' at: 2 is \$b. In the following we assume we are specifying a method in class **String** whose message selector is **min:**. The method returns a **String**, either the receiver of the message **min:** or its argument, whichever is colated first alphabetically.

min: aString

```
1 to: self size do:
  [ :index |
    (index > aString size) ifTrue: [aString].
    (self at: index) > (aString at: index) ifTrue: [aString].
    (self at: index) < (aString at: index) ifTrue: [self]].
↑self
```

The algorithm consists of two statements. The first is an iteration over each element of the receiver. The iteration stops when either (1) the argument, **aString**, no longer has a character with which to compare the next character in the receiver (i.e., **index > aString size**); (2) the next character in **self** comes after the next character in **aString** (i.e., **(self at: index) > (aString at: index)**); or (3) the next character in **self** comes before the next character in **aString**. As an example of (1), take the comparison of 'abcd' and 'abc' which terminates when **index = 4**; the answer is that 'abc' is first alphabetically. For (2), suppose we compare 'abde' with 'abce'. When **index = 3**, **\$d > \$c** is true; the answer is 'abce'. For (3), compare 'az' with 'by' which terminates when **index = 1**; the answer is 'az'. In the case that the receiver has fewer characters than the argument, even when the receiver is the initial substring of the argument, the first statement will complete and the second statement is evaluated; the result is the receiver. An example is the comparison of 'abc' and 'abcd'.

Note that arithmetic on characters is not supported. For example, the following expression is incorrect.

a ← \$A + 1

The error occurs because a **Character** does not understand the message **+**.

8

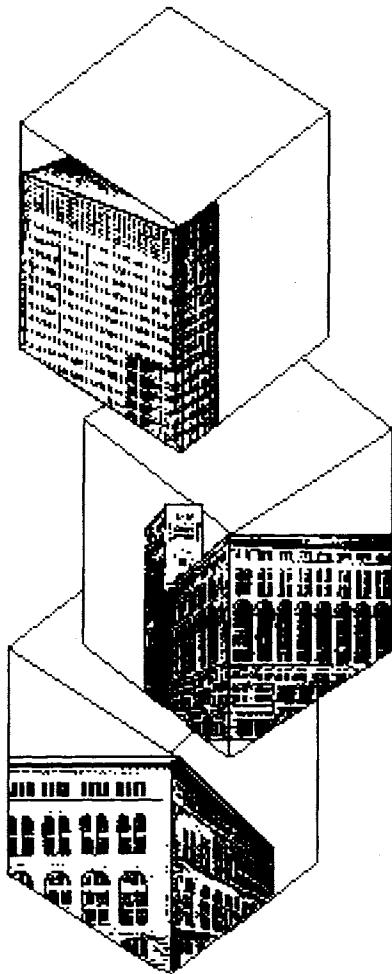
Numerical Classes

Protocol of the Number Classes

Classes Float and Fraction

Integer Classes

Class Random: A Random Number Generator



Object

Magnitude

Character
Date
Time

Stream

PositionableStream
ReadStream
WriteStream
ReadWriteStream
ExternalStream
FileStream

Number

Float
Fraction
Integer

LargeNegativeInteger
LargePositiveInteger
SmallInteger

Random

LookupKey

Association

File

FileDirectory
FilePage

UndefinedObject

Boolean
False
True

Link

Process

ProcessorScheduler

Delay
SharedQueue

Collection

SequenceableCollection
LinkedList

Behavior

ClassDescription
Class
MetaClass

Semaphore

ArrayedCollection
Array

Bitmap
DisplayBitmap

Point
Rectangle
BitBit

CharacterScanner

RunArray
String
Symbol

Pen

Text
ByteArray

DisplayObject

DisplayMedium
Form
Cursor
DisplayScreen

Interval
OrderedCollection
SortedCollection

InfiniteForm
OpaqueForm
Path

Bag
MappedCollection
Set

Dictionary
IdentityDictionary

Arc
Circle
Curve
Line
LinearFit
Spline

One of the major goals of the Smalltalk programming system is to apply a single metaphor for information processing as uniformly as possible. The Smalltalk metaphor, as described in earlier chapters, is one of objects that communicate by sending messages. This metaphor is very similar to the one used in Simula for implementing simulation systems. One of the greatest challenges to the application of the Smalltalk metaphor to all aspects of a programming system has been in the area of arithmetic. Simula used the object/message metaphor only for the higher level interactions in the simulations it implemented. For arithmetic, as well as most algorithmic control structures, Simula relied on the embedded Algol programming language with its built-in number representations, operations, and syntax. The contention that even the addition of two integers should be interpreted as message-sending met with a certain amount of resistance in the early days of Smalltalk. Experience has demonstrated that the benefits of this extreme uniformity in the programming language outweigh any inconvenience in its implementation. Over several versions of Smalltalk, implementation techniques have been developed to reduce the message-sending overhead for the most common arithmetic operations so that there is now almost no cost for the benefits of uniformity.

Objects that represent numerical values are used in most systems done in Smalltalk (as with most other programming languages). Numbers are naturally used to perform mathematical computations; they are also used in algorithms as indices, counters, and encodings of states or conditions (often called flags). Integral numbers are also used as collections of binary digits (bits) that perform boolean masking operations with each other.

Each different kind of numerical value is represented by a class. The number classes have been implemented so that all numbers behave as if they were of the most general type. The actual class of a particular number object is determined by how much of the full generality is needed to represent its value. Therefore the external protocol of all number objects is inherited from the class `Number`. `Number` has three subclasses: `Float`, `Fraction`, and `Integer`. `Integer` has three subclasses: `SmallInteger`, `LargePositiveInteger`, and `LargeNegativeInteger`. Integral numbers provide further protocol to support treating the number as a sequence of bits. This protocol is specified in the class `Integer`. Numbers in the system are instances of `Float`, `Fraction`, `SmallInteger`, `LargePositiveInteger`, or `LargeNegativeInteger`. Classes `Number` and `Integer` specify shared protocol, but they do not specify particular representations for numeric values. Therefore no instances of `Number` or `Integer` are created.

Unlike other objects that may change their internal state, the only state of a number is its value, which should never change. The object 3, for example, should never change its state to 4, or disastrous effects could occur.

Protocol of the Number Classes

Number defines the protocol of all numeric objects. Its messages support standard arithmetic operations and comparisons. Most of these must be implemented by subclasses of Number since they depend on the actual representation of values.

The protocol of arithmetic messages consists of the usual binary operators such as $+$, $-$, $*$ and $/$, and several unary and keyword messages for computing the absolute value of a number, the negation of a number, or the integer quotient or remainder of a number. The category for arithmetic messages is as follows.

Number instance protocol

arithmetic

$+$ aNumber	Answer the sum of the receiver and the argument, aNumber.
$-$ aNumber	Answer the difference between the receiver and the argument, aNumber.
$*$ aNumber	Answer the result of multiplying the receiver by the argument, aNumber.
$/$ aNumber	Answer the result of dividing the receiver by the argument, aNumber. Note that since as much precision as possible is retained, if the division is not exact, the result will be an instance of Fraction.
$//$ aNumber	Answer the integer quotient defined by division with truncation toward negative infinity.
$\backslash\backslash$ aNumber	Answer the integer remainder defined by division with truncation toward negative infinity. This is the modulo operation.
abs	Answer a Number that is the absolute value (positive magnitude) of the receiver.
negated	Answer a Number that is the negation of the receiver.
quo: aNumber	Answer the integer quotient defined by division with truncation toward zero.
rem: aNumber	Answer the integer remainder defined by division with truncation toward zero.
reciprocal	Answer 1 divided by the receiver. Report an error to the user if the receiver is 0.

Some examples follow.

<i>expression</i>	<i>result</i>
$1 + 10$	11
$5.6 - 3$	2.6
$5 - 2.6$	2.4
(-4) abs	4
$6 / 2$	3

7 / 2	(7/2), a Fraction with numerator 7 and de- nominator 2
7 reciprocal	(1/7), a Fraction with numerator 1 and de- nominator 7

Arithmetic messages that return integral quotients and remainders from a division operation follow two conventions. One convention truncates the resulting number toward zero, the other toward negative infinity. These are the same for positive results since zero and negative infinity are in the same direction. For negative results, the two conventions round in different directions. The protocol for Number provides for both conventions.

The following table shows the relationships among the selectors.

<i>result</i>	<i>truncate toward negative infinity</i>	<i>truncate toward zero</i>
quotient	//	quo:
remainder	\%	rem:

Examples include:

<i>expression</i>	<i>result</i>
6 quo: 2	3
7 quo: 2	3
(7 quo: 2) + 1	4
7 quo: 2 + 1	2
7 rem: 2	1
7 // 2	3
7 \% 2	1
7 \% 2 + 1	2
-7 quo: 2	-3
-7 rem: 2	-1
-7 // 2	-4
-7 \% 2	1

The result of quo:, rem:, or // is always to return a value whose sign is positive if the receiver and argument have the same sign, and negative if their signs are different. \% always produces a positive result.

Additional mathematical functions are

Number instance protocol

mathematical functions

exp	Answer a floating point number that is the exponential of the receiver.
ln	Answer the natural log of the receiver.
log: aNumber	Answer the log base aNumber of the receiver.
floorLog: radix	Answer the floor of the log base radix of the receiver, where the floor is the integer nearest the receiver toward negative infinity.
raisedTo: aNumber	Answer the receiver raised to the power of the argument, aNumber.
raisedToInteger: anInteger	Answer the receiver raised to the power of the argument, anInteger, where the argument must be a kind of Integer.
sqrt	Answer a floating point number that is the positive square root of the receiver.
squared	Answer the receiver multiplied by itself.

Some examples are

<i>expression</i>	<i>result</i>
2.718284 ln	1.0
6 exp	403.429
2 exp	7.38906
7.38906 ln	1.99998 (that is, 2)
2 log: 2	1.0
2 floorLog: 2	1
6 log: 2	2.58496
6 floorLog: 2	2
6 raisedTo: 1.2	8.58579
6 raisedToInteger: 2	36
64 sqrt	8.0
8 squared	64

Properties of numbers that deal with whether a number is even or odd and negative or positive can be tested with the following messages.

Number instance protocol

testing

even	Answer whether the receiver is an even number.
odd	Answer whether the receiver is an odd number.
negative	Answer whether the receiver is less than 0.
positive	Answer whether the receiver is greater than or equal to 0.
strictlyPositive	Answer whether the receiver is greater than 0.
sign	Answer 1 if the receiver is greater than 0, answer -1 if less than 0, else answer 0.

Properties of numbers that deal with truncation and round off are supplied by the following protocol.

Number instance protocol

truncation and round off

ceiling	Answer the integer nearest the receiver toward positive infinity.
floor	Answer the integer nearest the receiver toward negative infinity.
truncated	Answer the integer nearest the receiver toward zero.
truncateTo: aNumber	Answer the next multiple of the argument, aNumber, that is nearest the receiver toward zero.
rounded	Answer the integer nearest the receiver.
roundTo: aNumber	Answer the multiple of the argument, aNumber, that is nearest the receiver.

Whenever a Number must be converted to an Integer, the message truncated can be used. So we have

<i>expression</i>	<i>result</i>
16.32 ceiling	17
16.32 floor	16
-16.32 floor	-17
-16.32 truncated	-16
16.32 truncated	16
16.32 truncateTo: 5	15
16.32 truncateTo: 5.1	15.3
16.32 rounded	16
16.32 roundTo: 6	18
16.32 roundTo: 6.3	18.9

The protocol provided in class Number includes various messages for converting a number into another kind of object or a different unit of representation. Numbers can represent various unit measurements such as degrees and radians. The following two messages perform conversions.

Number instance protocol

converting

degreesToRadians	The receiver is assumed to represent degrees. Answer the conversion to radians.
radiansToDegrees	The receiver is assumed to represent radians. Answer the conversion to degrees.

So that

30 degreesToRadians = 0.523599
90 degreesToRadians = 1.5708

Trigonometric and logarithmic functions are included in the protocol for mathematical functions. The receiver for the trigonometric functions cos, sin, and tan is an angle measured in radians; the result of the functions arcCos, arcSin and arcTan is the angle measured in radians.

In the following examples, 30 degrees is given as 0.523599 radians; 90 degrees is 1.5708 radians.

<i>expression</i>	<i>result</i>
0.523599 sin	0.5
0.523599 cos	0.866025
0.523599 tan	0.57735
1.5708 sin	1.0
0.57735 arcTan	0.523551
1.0 arcSin	1.5708

When a kind of Integer is asked to add itself to another kind of Integer, the result returned will naturally also be a kind of Integer. The same is true for the sum of two Floats; the class of the result will be the same as the class of the operands. If the two operands are SmallIntegers and the absolute value of their sum is too large to be represented as a SmallInteger, the result will be a LargePositiveInteger or a LargeNegativeInteger. The determination of the appropriate class of result when the operands are of different classes is somewhat more complicated. Two design criteria are that there be as little loss of information as possible and that commutative operations produce the same result regardless of which operand is the receiver of the message and which is the argument. So for example, $3.1 * 4$ will return the same result as $4 * 3.1$.

The appropriate representation for the result of operations on numbers of different classes is determined by a numerical measure of generality assigned to each class. Classes said to have more generality will have a larger number for this generality measure. Each class must be able to convert its instances into equal-valued instances of more general classes. The measure of generality is used to decide which of the operands should be converted. In this way, the arithmetic operations obey the law of commutativity with no loss of numerical information. When the differences between two classes of numbers are only a matter of precision (where "precision" is a measure of the information provided in a number), the more precise class is assigned a higher degree of generality. We have arbitrarily assigned approximate numbers a higher generality in cases where precision was not the issue (so, Float is more general than Fraction).

The generality hierarchy for the kinds of numbers in the Smalltalk-80 system, with most general listed first, is

Float
 Fraction
 LargePositiveInteger, LargeNegativeInteger
 SmallInteger

The messages in the Number protocol designed to support the necessary coercions are categorized as "coercing" messages.

Number instance protocol

coercing

coerce: aNumber

Answer a number representing the argument, aNumber, that is the same kind of Number as the receiver. This method must be defined by all subclasses of Number.

generality

Answer the number representing the ordering of the receiver in the generality hierarchy.

retry: aSymbol coercing: aNumber

An arithmetic operation denoted by the symbol, aSymbol, could not be performed with the receiver and the argument, aNumber, as the operands because of the difference in representation. Coerce either the receiver or the argument, depending on which has the lower generality, and then try the arithmetic operation again. If the symbol is the equals sign, answer false if the argument is not a Number. If the generalities are the same, then retry:coercing: should not have been sent, so report an error to the user.

Thus if we try to evaluate $32.45 * 4$, the multiplication of a Float by a SmallInteger will result in evaluating the expression

`32.45 retry: #* coercing: 4`

and the argument 4 will be coerced to 4.0 (Float has higher generality than SmallInteger). Then the multiplication will be carried out successfully.

Defining a hierarchy of the numbers in terms of a numerical measure of generality works for the kinds of numbers provided in the basic Smalltalk-80 system because the generality is transitive for these kinds of numbers. However, it does not provide a technique that can be used for all kinds of numbers.

Intervals (described in detail in Chapter 10) can be created by sending one of two messages to a number. For each element of such an interval, a block can be evaluated with the element as the block value.

Number instance protocol

intervals

to: stop

Answer an Interval from the receiver up to the argument, stop, with each next element computed by incrementing the previous one by 1.

to: stop by: step

Answer an Interval from the receiver up to the argument, stop, with each next element computed by incrementing the previous one by step.

to: stop do: aBlock

Create an Interval from the receiver up to the argument, stop, incrementing by 1. Evaluate the argument, aBlock, for each element of the Interval.

to: stop by: step do: aBlock

Create an Interval from the receiver up to the argument, stop, incrementing by step. Evaluate the argument, aBlock, for each element of the Interval.

Thus if we evaluate

```
a ← 0.
```

```
10 to: 100 by: 10 do: [ :each | a ← a + each]
```

the final value of a will be 550.

If a is the array #('one' 'two' 'three' 'four' 'five'), then each element of the array can be accessed by indices that are in the interval from 1 to the size of the array. The following expression changes each element so that only the initial characters are kept.

```
1 to: a size do: [ :index | a at: index put: ((a at: index) at: 1)]
```

The resulting array is #('o' 't' 't' 'f' 'f'). Note that, like an array, elements of a string can be accessed using the messages at: and at:put:. Messages to objects like strings and arrays are detailed in Chapters 9 and 10.

Classes Float and Fraction

The classes Float and Fraction provide two representations of non-integral values. Floats are representations of real numbers that may be approximate; they represent about 6 digits of accuracy with a range between plus or minus 10 raised to the power plus or minus 32. Some examples are

```
8.0
```

```
13.3
```

```
0.3
2.5e6
1.27e-30
-12.987654e12
```

Fractions are representations of rational numbers that will always be exact. All arithmetic operations on a Fraction answer a reduced fractional result.

Instances of Float can be created by literal notation in methods (for example, 3.14159) or as the result of an arithmetic operation, one argument of which is another Float.

Instances of Fraction can be created as a result of an arithmetic operation if one of the operands is a Fraction and the other is not a Float. (If it were a Float, the result would be a Float since the generality number of Float is higher than that of Fraction). Instances of Fraction can also be created when the mathematical division operation (/) is performed on two Integers and the result is not integral. In addition, class protocol for Fraction supports sending a message of the form numerator: numInteger denominator: denInteger in order to create an instance.

Float responds to the message pi to return the corresponding constant. It adds truncation and round off protocol to return the fraction and integer parts of the receiver (fractionPart and integerPart), and it adds converting protocol to convert the receiver to a Fraction (asFraction). Similarly class Fraction adds converting protocol to convert the receiver to a Float (asFloat).

Integer Classes

Class Integer adds protocol particular to integral numbers. It has three subclasses. One is class SmallInteger, which provides a space-economical representation for a substantial range of integral values that occur frequently in counting and indexing. The representation limits the range to a little less than the magnitudes representable by a single machine word. Large integers, which are represented by instances of LargePositiveInteger or LargeNegativeInteger depending on the sign of the integer, do not have a limit to their magnitude. The cost in providing the generality of large integers is longer computation time. Thus if the result of an arithmetic operation on a large integer is representable as a small integer, it will in fact be a small integer.

In addition to the messages inherited from the class Number, class Integer adds converting protocol (asCharacter, asFloat and asFraction), further printing (printOn: aStream base: b, radix: baseInteger), and enumerating protocol. Thus 8 radix: 2 is 2r1000.

For enumerating, it is possible to evaluate a block repetitively an integral number of times using the message `timesRepeat: aBlock`. Take as an example

```
a ← 1.
10 timesRepeat: [a ← a + a]
```

where the block has no arguments. The resulting value of `a` is 2^{10} , or 1024.

Class `Integer` provides factorization and divisibility protocol not specified for numbers in general.

Integer instance protocol

factorization and divisibility

<code>factorial</code>	Answer the factorial of the receiver. The receiver must not be less than 0.
<code>gcd: anInteger</code>	Answer the greatest common divisor of the receiver and the argument, <code>anInteger</code> .
<code>lcm: anInteger</code>	Answer the least common multiple of the receiver and the argument, <code>anInteger</code> .

Examples are

<i>expression</i>	<i>result</i>
<code>3 factorial</code>	6
<code>55 gcd: 30</code>	5
<code>6 lcm: 10</code>	30

In addition to the numerical properties of integers, some algorithms make use of the fact that integers can be interpreted as a sequence of bits. Thus protocol for bit manipulation is specified in `Integer`.

Integer instance protocol

bit manipulation

<code>allMask: anInteger</code>	Treat the argument <code>anInteger</code> as a bit mask. Answer whether <i>all</i> of the bits that are 1 in <code>anInteger</code> are 1 in the receiver.
<code>anyMask: anInteger</code>	Treat the argument <code>anInteger</code> as a bit mask. Answer whether <i>any</i> of the bits that are 1 in <code>anInteger</code> are 1 in the receiver.
<code>noMask: anInteger</code>	Treat the argument <code>anInteger</code> as a bit mask. Answer whether <i>none</i> of the bits that are 1 in <code>anInteger</code> are 1 in the receiver.
<code>bitAnd: anInteger</code>	Answer an Integer whose bits are the logical <i>and</i> of the receiver's bits and those of the argument <code>anInteger</code> .
<code>bitOr: anInteger</code>	Answer an Integer whose bits are the logical <i>or</i> of the receiver's bits and those of the argument <code>anInteger</code> .

bitXor: anInteger	Answer an Integer whose bits are the logical <i>xor</i> of the receiver's bits and those of the argument anInteger.
bitAt: index	Answer the bit (0 or 1) at position index of the receiver.
bitInvert	Answer an Integer whose bits are the complement of the receiver.
highBit	Answer the index of the high order bit of the binary representation of the receiver.
bitShift: anInteger	Answer an Integer whose value (in two's-complement representation) is the receiver's value (in two's-complement representation) shifted left by the number of bits indicated by the argument, anInteger. Negative arguments shift right. Zeros are shifted in from the right in left shifts. The sign bit is extended in right shifts.

Some examples follow. Note that the default radix for printing an Integer is 10.

<i>expression</i>	<i>result</i>
2r111000111000111	29127
2r101010101010101	21845
2r101000101000101	20805
2r000111000111000	3640
29127 allMask: 20805	true
29127 allMask: 21845	false
29127 anyMask: 21845	true
29127 noMask: 3640	true
29127 bitAnd: 3640	0
29127 bitOr: 3640	32767
32767 radix: 2	2r111111111111111
29127 bitOr: 21845	30167
30167 radix: 2	2r111010111010111
3640 bitShift: 1	7280

Class Random: A Random Number Generator

Many applications require random choices of numbers. Random numbers are useful, for example, in statistical applications and data encryption algorithms. Class Random is a random number generator that is included in the standard Smalltalk-80 system. It provides a simple way of obtaining a sequence of random numbers that will be uniformly distributed over the interval between, but not including, 0.0 and 1.0.

An instance of class Random maintains a seed from which the next random number is generated. The seed is initialized in a pseudo-ran-

dom way. An instance of Random is sent the message next whenever a new random number is desired.

A random number generator can be created with the expression

```
rand ← Random new
```

The expression

```
rand next
```

can then be evaluated whenever a new random number is needed. The response is a number (Float) between 0.0 and 1.0.

The implementation of next is based on Lehmer's linear congruential method as presented in Knuth, Volume 1 [D. E. Knuth, *The Art of Computer Programming: Fundamental Algorithms*, Volume 1, Reading, Mass: Addison Wesley, 1968].

next

```
| temp |
"Lehmer's linear congruential method with modulus m = 2 raisedTo: 16,
a = 27181 odd, and 5 = a \\ 8, c = 13849 odd, and c / m approxi-
mately 0.21132"
[seed ← 13849 + (27181 * seed) bitAnd: 8r177777.
temp ← seed / 65536.0.
temp = 0] whileTrue.
↑temp
```

It is also possible to send an instance of class Random the messages next: anInteger, to obtain an OrderedCollection of anInteger number of random numbers, and nextMatchFor: aNumber, to determine whether the next random number is equal to aNumber.

Suppose we want to select one of 10 integers, 1, ..., 10, using the random number generator rand. The expression to be evaluated is

```
(rand next * 10) truncated + 1
```

That is,

<i>expression</i>	<i>result</i>
rand next	a random number between 0 and 1
rand next * 10	a random number between 0 and 10
(rand next * 10) truncated	an integer ≥ 0 and ≤ 9
(rand next * 10) truncated + 1	an integer ≥ 1 and ≤ 10